BÁO CÁO BÀI TẬP HỆ ĐIỀU HÀNH

OS_MEM_12 — Segmentation và địa chỉ logic

Ứng dụng desktop mô phỏng dịch địa chỉ, kiểm tra giới hạn và bảo vệ truy cập

Thông tin	Nội dung cần điền
Trường / khoa	_____
Học phần / lớp	_____
Giảng viên	_____
Mã nhóm	_____
Thành viên 1 — MSSV	_____
Thành viên 2 — MSSV	_____
Thành viên 3 — MSSV	_____
URL repository GitHub	https://github.com/tult842211/os-segmentation-simulator
URL video demo	_____
Ngày nộp thực tế	_____

Công nghệ: Python 3.10 trở lên, Tkinter, CSV, unittest. **Phạm vi:** một tiến trình, segment tăng theo chiều dương, một địa chỉ cho mỗi yêu cầu. **Mã nguồn:** các thư mục src/, tests/, scripts/. **Dữ liệu và minh chứng:** input/, output/.

Lưu ý hoàn thiện hồ sơ

Đây là bản báo cáo đi kèm chương trình và dữ liệu có thể chạy lại. Các dòng trống dành cho thông tin thật của nhóm. Bảng kế hoạch được ghi là đề xuất; không đại diện cho lịch sử thực hiện. Nhóm cần điền đóng góp thực tế, tự quay video và kiểm tra quyền truy cập trước khi nộp.

Các bảng giá trị kỳ vọng trong báo cáo được tính từ định nghĩa mô hình. Kết quả thực chạy và thời gian đo được lưu ở output/; số liệu không được tự suy ra từ độ phức tạp lý thuyết. Nhóm phải phân biệt hai loại minh chứng này khi trình bày.

1. Bài toán, mục tiêu và phạm vi

1.1. Bài toán cần giải quyết

Trong Topic 12, đầu vào gồm một bảng segment và các địa chỉ logic. Với mỗi yêu cầu, chương trình cần xác định segment có tồn tại, offset có nằm trong giới hạn và thao tác có được phép hay không. Nếu tất cả điều kiện đều đúng, kết quả là địa chỉ vật lý. Nếu một điều kiện sai, kết quả chỉ rõ nguyên nhân và dừng trước bước tạo địa chỉ hợp lệ.

Mục tiêu học tập của sản phẩm là làm rõ từng quyết định trên một ví dụ quan sát được. Người dùng có thể nhập một địa chỉ, xem từng bước, chạy một danh sách CSV, đối chiếu với phép tính tay và kiểm tra dữ liệu xuất. Phần lõi độc lập với giao diện để cùng một thuật toán được dùng cho demo, kiểm thử và đo hiệu năng.

1.2. Cơ sở lý thuyết

Segmentation chia không gian địa chỉ thành những vùng logic có độ dài riêng, chẳng hạn mã lệnh và dữ liệu. Mỗi vùng có thông tin vị trí bắt đầu, giới hạn và quyền. Địa chỉ gồm phần chọn segment cùng độ lệch trong vùng đó. Khái niệm này và các vấn đề bảo vệ, chia sẻ, phân mảnh ngoài được trình bày trong chương Segmentation của OSTEP [1]. Bài tập đặt trong chủ đề quản lý bộ nhớ của học phần Hệ điều hành; tài liệu tham khảo bổ sung là trang sách Operating System Concepts [2].

Bản mô phỏng chọn mô hình đơn giản để công thức, lỗi và phép kiểm tra có thể được tính bằng tay. Nó không mô tả toàn bộ cơ chế quản lý bộ nhớ của một hệ điều hành đang chạy.

1.3. Các sản phẩm cần kiểm chứng

Mã	Nội dung báo cáo	Minh chứng trong dự án
Q1	Trạng thái, đầu vào, giả định	Mục 2; các lớp trong segmentation.py
Q2	Pseudocode và trace	Mục 3–4; chế độ từng bước
Q3	Ca bình thường, biên, lỗi; expected/actual	Mục 4, 7; verification.csv
Q4	GUI, CSV, tách module	Mục 5–6; chương trình desktop
Q5	Độ phức tạp và so sánh định lượng	Mục 8; benchmark.csv
Q6	Phối hợp, kiểm thử, giới hạn	Mục 7, 9–10; bảng đóng góp

1.4. Tiêu chí hoàn thành

Một yêu cầu thành công phải cho đúng tổng base + offset, đồng thời không vượt vùng cho phép. Mỗi yêu cầu bị từ chối phải có một trạng thái xác định và không có địa chỉ vật lý. CSV xuất phải giữ được kết quả và trace để kiểm tra sau khi đóng ứng dụng. Chương trình cần mở được từ thư mục dự án bằng hướng dẫn trong README, không yêu cầu sửa đường dẫn cá nhân trong mã nguồn.

Phần thông tin sinh viên, nhận xét của giảng viên, đóng góp nhóm và video không thể được thay bằng dữ liệu mẫu. Chúng được hoàn thiện từ quá trình sử dụng và kiểm chứng thật của nhóm.

2. Mô hình trạng thái, dữ liệu và giả định — Q1

2.1. Dữ liệu segment và địa chỉ

Mỗi Segment có bốn thuộc tính: segment_id, base, limit, permissions. ID và base là số nguyên không âm; limit là số nguyên dương. Quyền là một chuỗi không rỗng chỉ chứa các ký tự r, w, x, mỗi ký tự xuất hiện tối đa một lần. ID trong bảng phải duy nhất. Bảng phải có ít nhất một segment.

Một LogicalAddress gồm ID không âm, offset nguyên và access là đúng một ký tự r, w hoặc x. Offset có thể âm trong dữ liệu yêu cầu để thử một truy cập bất hợp lệ; nó không thể tạo ra kết quả thành công. Quyền mặc định là rw cho segment và thao tác mặc định là r cho địa chỉ khi các cột tùy chọn bị bỏ khỏi CSV.

Ký hiệu	Diễn giải	Ví dụ
s	Mã segment	0
b	Base, địa chỉ bắt đầu	1000
l	Limit, số vị trí trong vùng	400
d	Offset, vị trí tính từ base	120
a	Thao tác được yêu cầu	r
p	Địa chỉ vật lý nếu hợp lệ	1120

2.2. Miền địa chỉ và quy tắc không chồng lấn

Segment chiếm khoảng nửa mở [b, b + l). Với ví dụ trên, địa chỉ đầu là 1000, địa chỉ cuối là 1399; không có địa chỉ 1400 trong segment này. Tập offset hợp lệ là các số nguyên từ 0 đến 399. Dùng khoảng nửa mở giúp phân biệt độ dài với chỉ số cuối và xử lý hai vùng nằm sát nhau.

Trong mô hình một tiến trình của dự án, các khoảng segment không chồng lấn. Sắp xếp theo base, rồi kiểm tra từng cặp kề nhau. Nếu base của vùng sau nhỏ hơn base + limit của vùng trước, bảng bị từ chối. Nếu bằng, hai vùng tiếp giáp và được chấp nhận. Việc cấm chồng lấn là giả định của mô hình này; chia sẻ vùng nhớ giữa nhiều tiến trình nằm ngoài phạm vi.

2.3. Kết quả và trạng thái

Trạng thái	Điều kiện	Địa chỉ vật lý
SEGMENT_NOT_FOUND	ID không có trong bảng	Không có
SEGMENTATION_FAULT	Segment tồn tại nhưng d < 0 hoặc d >= l	Không có
PROTECTION_FAULT	Offset hợp lệ nhưng access không thuộc permissions	Không có
OK	Tất cả điều kiện đạt	b + d

TranslationResult giữ đầu vào, base/limit nếu tìm thấy, trạng thái, địa chỉ, thông báo và trace. Lỗi định dạng như offset = abc, ID âm hoặc quyền q bị chặn ở lớp dữ liệu; chúng không phải một trong các kết quả dịch hợp lệ về định dạng.

Địa chỉ là số nguyên trừu tượng của Python, không mặc định 32-bit hay 64-bit. Mô hình không đọc/ghi byte thật, không mô phỏng cấp phát động và không thay đổi bảng trong một lần dịch. Một kết quả lỗi trong ứng dụng không gây segmentation fault thật cho hệ điều hành.

3. Thuật toán, pseudocode và bất biến — Q2

3.1. Chuẩn bị bảng

```
BUILD_TABLE(segments):
    yêu cầu bảng không rỗng
    index = dictionary rỗng
    với mỗi segment:
        kiểm tra ID, base, limit, permissions
        nếu ID đã có trong index: báo lỗi bảng
        index[ID] = segment
    ordered = sắp xếp segments tăng theo base
    với mỗi cặp previous, current kề nhau:
        nếu current.base < previous.base + previous.limit:
            báo lỗi chồng lấn
    trả bảng gồm danh sách segment và index
```

3.2. Dịch một địa chỉ

```
TRANSLATE(table, address):
    trace = ["Tra bảng theo ID"]
    segment = table.index.get(address.segment_id)
    nếu segment không tồn tại:
        trả SEGMENT_NOT_FOUND, không có địa chỉ, trace
    ghi base, limit và permissions vào trace
    nếu address.offset < 0 hoặc address.offset >= segment.limit:
        trả SEGMENTATION_FAULT, không có địa chỉ, trace
    ghi kết quả kiểm tra giới hạn vào trace
    nếu address.access không thuộc segment.permissions:
        trả PROTECTION_FAULT, không có địa chỉ, trace
    ghi kết quả kiểm tra quyền vào trace
    physical = segment.base + address.offset
    ghi phép cộng vào trace
    trả OK, physical, trace
```

Luồng điều khiển có thể đọc theo thứ tự: **nhập → tìm segment → kiểm tra giới hạn → kiểm tra quyền → cộng địa chỉ → xuất kết quả**. Mỗi nhánh lỗi kết thúc phép dịch hiện tại; batch vẫn xử lý các địa chỉ còn lại. Thứ tự kiểm tra tạo kết quả nhất quán khi một yêu cầu đồng thời sai nhiều điều kiện.

3.3. Vì sao công thức đúng trong mô hình

Sau khi vượt qua kiểm tra giới hạn, ta có $0 <= d < l$. Cộng b vào ba vế cho $b <= b + d < b + l$. Do đó địa chỉ trả về nằm đúng trong khoảng segment đã chọn. Kiểm tra quyền bảo đảm phép truy cập yêu cầu thuộc quyền được khai báo. Hai điều kiện cùng cần thiết: nằm đúng vùng nhưng ghi vào vùng chỉ đọc vẫn phải bị từ chối.

Bất biến của kết quả thành công là $physical = base + offset$, $base <= physical < base + limit$ và access được phép. Bất biến của kết quả lỗi là $physical_address = None$. Địa chỉ lỗi không được mang một giá trị tính thử có thể bị hiểu nhầm là địa chỉ hợp lệ.

Thuật toán không thay đổi bảng. Vì vậy, cùng bảng và cùng địa chỉ cho cùng trạng thái, địa chỉ vật lý và trình tự bước. Tính xác định này cho phép tính tay expected trước, chạy lại test và so sánh dữ liệu mà không phụ thuộc thứ tự các yêu cầu trong batch.

4. Ví dụ tính tay, trace và oracle — Q2, Q3

4.1. Bảng dùng trong ví dụ

ID	Base	Limit	Quyền	Khoảng vật lý hợp lệ
0	1000	400	rx	1000–1399
1	2000	300	rw	2000–2299
2	3000	500	rw	3000–3499

Ba khoảng tách rời, ID không trùng và limit dương, nên bảng đáp ứng giả định. Đây cũng là bảng trong input/segments.csv.

4.2. Trace của yêu cầu (0, 120, r)

1. Tra bảng với ID 0, tìm thấy segment.
2. Đọc base = 1000, limit = 400, permissions = rx.
3. Kiểm tra 0 <= 120 < 400: đúng.
4. Kiểm tra thao tác r có trong rx: đúng.
5. Tính 1000 + 120 = 1120, trả OK.

Trace của (0, 400, r) giống hai bước đầu, nhưng dừng ở kiểm tra 0 <= 400 < 400, vốn sai. Không thực hiện bước trả một địa chỉ vật lý. Với (0, 100, w), giới hạn đúng nhưng w không có trong rx, nên dừng ở kiểm tra quyền.

4.3. Giá trị kỳ vọng được tính độc lập

Dòng input	Yêu cầu (ID, offset, access)	Trạng thái kỳ vọng	Địa chỉ kỳ vọng
1	(0, 120, r)	OK	1120
2	(0, 0, x)	OK	1000
3	(0, 399, r)	OK	1399
4	(0, 400, r)	SEGMENTATION_FAULT	—
5	(0, -1, r)	SEGMENTATION_FAULT	—
6	(1, 50, w)	OK	2050
7	(2, 499, r)	OK	3499
8	(9, 10, r)	SEGMENT_NOT_FOUND	—
9	(0, 100, w)	PROTECTION_FAULT	—
10	(1, 20, x)	PROTECTION_FAULT	—

Các dòng trên bao gồm ca bình thường, biên đầu/cuối, ngay ngoài biên, offset âm, thiếu segment và sai quyền. input/expected_results.csv giữ các giá trị kỳ vọng này. Script scripts/verify.py chạy chương trình trên input, đối chiếu từng yêu cầu với expected rồi ghi actual và kết quả so sánh vào output/verification.csv.

Không tạo expected bằng cách gọi chính translate(), vì làm vậy có thể lặp lại cùng một lỗi và cho cảm giác đã kiểm chứng. Khi trình bày, nhóm mở verification.csv để đọc actual của ít nhất ba nhóm ca; dấu gạch trong bảng này chỉ "không có địa chỉ", không phải kết quả thực chạy đã được điền sẵn.

5. Kiến trúc phần mềm và trách nhiệm module — Q4

5.1. Phân chia trách nhiệm

Thành phần	Trách nhiệm	Đầu ra chính
run.py, src/main.py	Khởi tạo ứng dụng	Cửa sổ Tkinter
src/segmentation.py	Mô hình, kiểm tra bảng, dịch một/nhiều địa chỉ	TranslationResult
src/csv_io.py	Đọc CSV và kiểm tra cấu trúc; xuất kết quả	Đối tượng dữ liệu hoặc lỗi rõ ràng
src/gui.py	Bảng, biểu mẫu, trace, sơ đồ và tương tác	Trình bày kết quả lõi
tests/	Kiểm thử logic và nhập/xuất	Kết quả pass/fail
scripts/verify.py	Đối chiếu bộ mẫu với oracle	verification.csv
scripts/benchmark.py	Chạy phép đo và tổng hợp	Dữ liệu, tóm tắt hiệu năng

Luồng dữ liệu từ file là: **CSV → kiểm tra cấu trúc → tạo Segment/LogicalAddress → SegmentTable → TranslationResult → GUI hoặc CSV xuất**. Với nhập tay, biểu mẫu thay cho bước đọc CSV; các đối tượng lõi vẫn áp dụng cùng điều kiện dữ liệu. Các công cụ kiểm chứng gọi cùng phần lõi mà không cần mở cửa sổ.

5.2. Dữ liệu bất biến và tính nhất quán

Các bản ghi segment, địa chỉ và kết quả dùng dataclass bất biến. Bảng giữ một tập segment và một chỉ mục dictionary theo ID. Khi người dùng chỉnh segment, ứng dụng xây dựng và kiểm tra bảng mới; một bảng có cấu trúc sai không được sử dụng như bảng hợp lệ.

Một batch cần gắn với bảng đã dùng lúc chạy. Nếu người dùng thay đổi bảng sau đó, kết quả cũ vẫn phải được hiểu theo snapshot của batch. Sơ đồ của dòng batch được chọn dùng chính bảng gắn với kết quả đó. Điều này tránh diễn giải một kết quả theo base/limit mới, khiến trace và sơ đồ không khớp nhau.

5.3. Công việc nền và luồng giao diện

Đọc file và dịch một danh sách dài có thể mất thời gian. GUI sử dụng công việc nền và hàng đợi kết quả; vòng lặp giao diện thăm dò hàng đợi để cập nhật widget trên luồng chính. Các nút liên quan được quản lý trạng thái trong khi tác vụ chạy. Phần giao diện dùng cơ chế Tkinter được mô tả trong tài liệu chính thức [3].

Việc xử lý nền làm cửa sổ có thể phản hồi tốt hơn trong khi chờ; nó không đồng nghĩa thuật toán CPU tự động nhanh hơn. Số liệu hiệu năng phần lõi phải được đo riêng, không lấy tốc độ thay đổi widget làm tốc độ dịch địa chỉ.

5.4. Lợi ích khi sửa và kiểm tra

Nếu cần thay bố cục, thay màu hoặc thêm một trường hiển thị, thay đổi chủ yếu nằm ở gui.py. Nếu cần sửa điều kiện biên, thay đổi nằm ở lõi và được kiểm tra bằng unit test trước khi mở GUI. CSV dùng kết quả đã tạo từ lõi, không tự cài một công thức dịch khác. Cách chia module này làm giảm nguy cơ GUI hiển thị đúng nhưng script kiểm chứng hoặc file xuất dùng thuật toán khác.

6. Thiết kế GUI và hợp đồng CSV — Q4

6.1. Các thao tác chính trên giao diện

Tab **Dịch địa chỉ** hiển thị bảng segment, vùng thêm/cập nhật/xóa segment, địa chỉ logic, quyền truy cập, kết quả, trace và sơ đồ. Nút **Nạp mẫu** nạp bảng mẫu; **Mở segments.csv** cho chọn một bảng khác. **Dịch địa chỉ** hiển thị kết quả đầy đủ; **Từng bước** và **Bước tiếp** → lần lượt mở các bước của trace để phục vụ giảng giải.

Tab **Xử lý CSV** có **Mở addresses.csv và chạy**, **Chạy bộ địa chỉ mẫu**, bảng kết quả và **Xuất kết quả CSV**. Chọn một dòng để xem trace; **Xem dòng chọn trên sơ đồ** → giúp đối chiếu với vùng nhớ. Tab **Hướng dẫn** giải thích trường nhập, công thức và cách sử dụng.

Bảng GUI hiển thị tối đa 1.000 kết quả đầu để hạn chế chi phí tạo widget; file xuất chứa toàn bộ kết quả đã xử lý. Sơ đồ trình bày tối đa 10 segment và có segment đang chọn. Thanh dùng cùng thang địa chỉ, nhưng vùng quá nhỏ được vẽ rộng tối thiểu để nhìn thấy; cần đọc con số và không đo độ dài bằng pixel để tính địa chỉ.

6.2. Hai hợp đồng đầu vào

```
segment_id,base,limit,permissions
0,1000,400,rx
1,2000,300,rw
```

```
segment_id,offset,access
0,120,r
0,400,r
0,100,w
```

Cột permissions có thể bị bỏ hoàn toàn để dùng mặc định rw; access có thể bị bỏ để dùng r. Nếu đã có cột, ô dữ liệu phải không rỗng. Header phải đúng tên, không trùng, không thiếu cột bắt buộc và không có cột ngoài hợp đồng. Dấu phân cách là dấu phẩy. Số nguyên viết ở hệ thập phân; chuỗi như 1.5 hoặc 0x10 không được coi là số nguyên CSV hợp lệ.

6.3. Kiểm tra và xuất dữ liệu

CSV rỗng, chỉ có header, số ô không khớp, tên cột sai, ô trống, số sai kiểu hoặc quyền không hợp lệ đều phải được báo lỗi. Khi lỗi gắn với dữ liệu một dòng, thông báo có số dòng để người dùng sửa. Sau kiểm tra từng dòng segment còn có kiểm tra toàn bảng: ID trùng và vùng chồng lấn.

File kết quả có các cột segment_id, offset, access, base, limit, status, physical_address, message, trace. Trace nằm trong cùng CSV; các bước ngăn bằng xuống dòng bên trong ô và được thư viện CSV trích dẫn đúng định dạng. Kết quả lỗi có ô địa chỉ vật lý trống. File đọc hỗ trợ UTF-8 có hoặc không BOM; file xuất dùng UTF-8 BOM. Đọc/ghi dựa trên thư viện csv, không tự tách chuỗi theo dấu phẩy [4].

7. Chiến lược kiểm thử và minh chứng đúng/sai — Q3, Q6

7.1. Ba lớp kiểm thử

Kiểm thử đơn vị kiểm tra mô hình và từng nhánh dịch: tạo bảng hợp lệ, từ chối tham số sai, giới hạn đầu/cuối, thiếu segment, quyền và sự nhất quán của trace. Các trường hợp cần đặc biệt chú ý là limit bằng 1, hai segment tiếp giáp, ID thưa và yêu cầu vừa vượt biên vừa sai quyền. Công cụ sử dụng unittest của Python [5].

Kiểm thử nhập/xuất dùng file tạm để kiểm tra UTF-8/BOM, cột tùy chọn, header sai, thiếu ô, số sai kiểu, dữ liệu rỗng và đọc lại kết quả có trace nhiều dòng. Một file có dòng lỗi không nên bị âm thầm bỏ qua rồi được báo như xử lý đủ dữ liệu.

Kiểm tra tích hợp và GUI chạy bộ mẫu qua CSV, đối chiếu oracle và thao tác cửa sổ. Nhóm cần nạp bảng, dịch một địa chỉ, chạy batch, chọn dòng, xem sơ đồ, sửa bảng và xuất file. Unit test thành công không tự chứng minh cỡ chữ, sơ đồ hoặc hộp thoại đều dễ dùng trên máy trình bày.

7.2. Ma trận các nhóm ca cần kiểm chứng

Nhóm	Dữ liệu đại diện	Kỳ vọng
Bình thường	(0, 120, r) với bảng mẫu	OK, 1120
Biên đầu	(0, 0, x)	OK, 1000
Biên cuối	(0, 399, r)	OK, 1399
Vượt biên	(0, 400, r)	SEGMENTATION_FAULT
Âm	(0, -1, r)	SEGMENTATION_FAULT
Không tồn tại	(9, 10, r)	SEGMENT_NOT_FOUND
Sai quyền	(0, 100, w)	PROTECTION_FAULT
Sai cấu hình	ID trùng; limit 0; vùng chồng lấn	Từ chối bảng
Sai định dạng	Thiếu header; offset abc	Báo lỗi đầu vào

7.3. Cách tái lập và đọc kết quả thực tế

```
py -m unittest discover -s tests -v
py scripts/verify.py
```

Lần chạy ngày 04/10/2026 được lưu trong output/test-results.txt đạt **69/69 kiểm thử**, gồm 13 kiểm thử GUI và 3 kiểm thử cập nhật số liệu báo cáo, không bỏ qua ca nào; thời gian **1,875 giây**, trên **Windows với Python 3.12.14**. output/verification.csv ghi expected/actual cho từng yêu cầu mẫu; cần xem cả trạng thái và địa chỉ. Nếu sửa lõi hoặc dữ liệu, phải chạy lại kiểm tra liên quan trước khi dùng minh chứng cũ.

Khi có sai khác, giữ input tái hiện, tính lại expected bằng tay, xác định lỗi ở đọc file hay dịch địa chỉ rồi sửa. Bổ sung test hồi quy cho nguyên nhân đã tìm thấy. Không sửa expected để khớp actual nếu chưa chứng minh expected cũ sai. Kết quả thực nghiệm cùng các file nguồn kiểm tra là bằng chứng có thể xem lại, không thay cho việc hiểu các điều kiện đúng/sai.

8. Độ phức tạp và so sánh hiệu năng — Q5

8.1. Phân tích lý thuyết

Gọi S là số segment và A là số địa chỉ. Xây dictionary tốn thời gian kỳ vọng O(S). Sắp xếp segment để kiểm tra chồng lấn tốn O(S log S), sau đó duyệt O(S). Tổng chi phí tạo bảng vì vậy là O(S log S) theo mô hình thông thường. Tra ID qua dictionary có chi phí trung bình O(1); các kiểm tra biên/quyền và phép cộng có số thao tác cố định khi xem độ dài số là hữu hạn.

Công việc	Dictionary	Duyệt tuyến tính
Một lần tìm ID	Trung bình O(1)	Xấu nhất O(S)
Tìm cho A địa chỉ	Kỳ vọng O(A)	Xấu nhất O(A × S)
Lưu bảng	O(S)	O(S)
Lưu A kết quả/trace	O(A)	Tùy cách tạo kết quả

Dictionary không được khẳng định luôn O(1) trong mọi tình huống. Địa chỉ Python rất lớn còn làm chi phí chuyển đổi chuỗi, phép toán và định dạng phụ thuộc số chữ số. Phân tích trên dùng kích thước số thông thường của các bộ dữ liệu thực nghiệm.

8.2. Thiết kế phép đo công bằng

Benchmark cần phân biệt tra cứu ID thuần túy với dịch đầy đủ có tạo đối tượng kết quả và trace. Với so sánh tra cứu, cả dictionary và tuyến tính nhận cùng bảng, cùng danh sách ID và cùng yêu cầu trả về. Với đo dịch đầy đủ, phần tạo dữ liệu và đọc/ghi CSV nằm ngoài khoảng thời gian đang đánh giá. Không so một phía chỉ get() với phía phải tạo trace rồi kết luận đó là mức tăng tốc của cả ứng dụng.

Các bộ đo thay đổi số segment và số địa chỉ, chạy lặp và ghi nhận môi trường. Script sử dụng đồng hồ độ phân giải cao perf_counter/perf_counter_ns; thời lượng là hiệu hai mốc [6]. Đây không phải phép đo chu kỳ bộ nhớ phần cứng.

8.3. Kết quả định lượng được lưu cùng dự án

Chạy py scripts/benchmark.py. Dữ liệu ở output/benchmark.csv, tóm tắt ở output/benchmark-summary.md, môi trường ở output/system-info.json. Trích số liệu phải kèm cấu hình, đơn vị và loại phép đo. Thời gian máy khác có thể khác; không chỉnh số liệu để khớp nhận định lý thuyết.

8.4. Kết quả thực nghiệm

Đo ngày 03/10/2026 trên **CPython 3.12.14, Windows 11, AMD64**, CPU được hệ điều hành nhận diện là Intel64 Family 6 Model 154 Stepping 3, GenuineIntel. Đây là số đo của môi trường thực nghiệm này; thời gian trên máy khác hoặc lần chạy khác có thể thay đổi.

Đối chiếu tính đúng đắn

Phép kiểm chứng	Kết quả thực tế
Bộ CSV mẫu đối chiếu với expected tính tay	**10/10 PASS**
Stress: 100.000 địa chỉ trên 1.000 segment	**PASS**, checksum và số trạng thái khớp công thức độc lập
Thời gian stress, một lượt streaming	**331,918 ms**
Trạng thái trong stress	OK: 39.520; không có segment: 9.940; vượt biên: 18.050; sai quyền: 32.490

Stress dùng lặp một bộ 10.000 địa chỉ đã tạo sẵn và không giữ toàn bộ kết quả/trace trong bộ nhớ. Thời gian stress bao gồm dịch địa chỉ, cập nhật số lượng trạng thái và checksum; không phải median và không trực tiếp tương đương phép đo batch bên dưới.

Đo batch với 1.000 segment

Mỗi cấu hình dùng seed 12, chạy warm-up một lượt rồi lấy **median 5 lượt** bằng time.perf_counter. Input và bảng được chuẩn bị trước phép đo dịch. Đơn vị trong bảng là **mili giây (ms)**; dấu chấm là dấu thập phân.

Số địa chỉ	Dựng bảng	Dịch đầy đủ và trace	Chỉ tra cứu dict	Chỉ tra cứu tuyến tính
100	0.1388	0.2631	0.0074	0.8711
1.000	0.1308	2.6268	0.0701	9.6649
10.000	0.1326	32.2393	0.7285	96.3816

Dựng bảng gồm lập chỉ mục và kiểm tra chồng lấn, không gồm tạo đối tượng Segment. Dịch đầy đủ gồm kiểm tra giới hạn/quyền và tạo kết quả/trace; không gồm CSV hay GUI. Hai cột tra cứu dùng cùng danh sách ID và phép cộng checksum, đã kiểm tra đối tượng trả về tương đương trước khi đo. Tỷ lệ giữa hai cột này chỉ mô tả phần tra cứu.

Ở cấu hình 1.000 segment / 10.000 địa chỉ, peak cấp phát Python được tracemalloc ghi nhận cho một lượt dịch riêng là **8.735.226 byte**. Chỉ số này gồm kết quả/trace; không phải RAM toàn tiến trình và không gồm input/bảng đã tồn tại.

Nguồn số liệu trong repository: output/verification.csv, output/benchmark.csv, output/system-info.json. Phương pháp đầy đủ: output/benchmark-summary.md. Mã tái lập: scripts/verify.py và scripts/benchmark.py. Bảng này được tự động tạo lại khi benchmark hoàn tất; có thể cập nhật từ số đo đã lưu bằng python scripts/report_metrics.py.

9. Quy trình GitHub, nhóm và khả năng tái lập — Q6

9.1. Chạy từ một bản dự án sạch

Máy cần Python 3.10 trở lên có Tkinter. Kiểm tra bằng py --version, py -m tkinter; sau đó chạy py run.py tại thư mục gốc. Không có gói bên ngoài bắt buộc cho ứng dụng và unit test. Có thể dùng lệnh python nếu môi trường không cài Windows Python Launcher.

Một người kiểm tra độc lập nên tải ZIP/clone mới, chạy ứng dụng, nạp mẫu, chạy test, xác minh và benchmark theo README. Việc này phát hiện phụ thuộc ẩn vào file cá nhân hoặc thư mục tạm. Tài liệu môi trường ghi phiên bản dùng trong phép đo, còn README nêu mức phiên bản mà dự án đặt mục tiêu hỗ trợ.

9.2. Sử dụng GitHub trong quá trình học

Repository lưu mã nguồn, CSV, tài liệu và minh chứng. Nhóm tạo issue cho công việc thực tế, tạo nhánh cho thay đổi, chạy kiểm tra rồi mở pull request để người khác xem. Commit nên mô tả thay đổi cụ thể, chẳng hạn sửa điều kiện biên hoặc thêm test chồng lấn. Không tạo nhiều commit lùi ngày để giả thành quá trình tám tuần.

Repository riêng tư cần cấp quyền theo cách giảng viên yêu cầu. URL trong báo cáo không bảo đảm người chấm mở được. Video phải được tự quay, tải lên nơi được chấp nhận và thử quyền xem; một file kịch bản không thay thế video.

9.3. Kế hoạch đề xuất và đóng góp thực tế

Giai đoạn đề xuất	Nội dung
Tuần 1–2	Đọc đề, hiểu mô hình, tính tay, kiểm tra phần lõi
Tuần 3–4	CSV, GUI, tập thao tác và kiểm tra chéo
Tuần 5–6	Kiểm thử mở rộng, đo hiệu năng, phân tích
Tuần 7–8	Hoàn thiện báo cáo, tập bảo vệ và quay video

Thành viên — MSSV	Đóng góp thực tế	Minh chứng
_____	_____	_____
_____	_____	_____
_____	_____	_____

Bảng đầy đủ và nhật ký trống ở report/team-plan.md. Chỉ điền sau khi thực hiện. Mỗi thành viên cần giải thích được điều kiện biên, một nhánh lỗi, cấu trúc dữ liệu và một bảng số liệu; việc chia vai trò không loại bỏ trách nhiệm hiểu bài chung.

9.4. Khai báo AI

Codex hỗ trợ xây dựng mã nguồn, kiểm thử, công cụ thực nghiệm và bản thảo tài liệu. Nhóm chịu trách nhiệm kiểm tra, chạy lại, bổ sung nhận xét thực tế và hiểu nội dung khi bảo vệ. Phạm vi hỗ trợ và mẫu xác nhận ở report/ai-disclosure.md; không trình bày phần AI tạo như bằng chứng một cá nhân tự thực hiện toàn bộ.

10. Giới hạn, hướng phát triển và tài liệu tham khảo

10.1. Giới hạn của kết quả

Dự án mô phỏng một tiến trình và các vùng không chồng lấn. Nó chưa mô phỏng chia sẻ segment, chuyển ngữ cảnh giữa tiến trình, stack tăng xuống, paging, TLB hoặc cấp phát/thu hồi động. Các khoảng trống trên sơ đồ là khoảng giữa các segment đã cho, không phải kết quả đánh giá một bộ cấp phát. Thông báo lỗi là dữ liệu mô phỏng, không phải trap phần cứng.

Mỗi yêu cầu biểu diễn một địa chỉ. Nếu bổ sung truy cập nhiều byte, phải kiểm tra toàn bộ khoảng, chẳng hạn offset + length <= limit, và điều kiện của length. Nếu mô phỏng địa chỉ 32-bit, phải bổ sung cận trên và quy tắc tràn. Hiện tại Python dùng số nguyên tùy ý; điều đó giúp ví dụ đơn giản nhưng không phản ánh thanh ghi có độ rộng cố định.

GUI giới hạn số dòng và số segment vẽ để giữ khả năng đọc; CSV xuất là nơi xem toàn bộ kết quả. Bộ nhớ của batch tăng theo dữ liệu và trace vì chương trình lưu danh sách; streaming là một hướng cải tiến nếu cần xử lý file lớn hơn. Mọi kết luận tốc độ chỉ áp dụng cho cấu hình và điều kiện được đo.

10.2. Hướng phát triển có thể kiểm chứng

Có thể thêm nhiều tiến trình với bảng riêng và quyền chia sẻ rõ ràng; thêm độ dài thao tác để kiểm tra toàn khoảng; hoặc thêm một mô hình cấp phát động có chuỗi yêu cầu và chỉ số phân mảnh xác định. Mỗi mở rộng cần cập nhật giả định, CSV, test và báo cáo. Với mục tiêu hiện tại, ưu tiên làm rõ và kiểm chứng dịch địa chỉ trước khi bổ sung nhiều cơ chế khác.

10.3. Tài liệu tham khảo

Các nguồn được kiểm tra ngày 03/10/2026. Thông tin chi tiết về cách dùng nguồn nằm trong report/references.md.

1. R. H. Arpaci-Dusseau, A. C. Arpaci-Dusseau. Operating Systems: Three Easy Pieces, Chapter 16, Segmentation. PDF chính thức, University of Wisconsin–Madison.

2. A. Silberschatz, P. B. Galvin, G. Gagne. Operating System Concepts, 10th edition. Trang sách chính thức của tác giả.

3. Python Software Foundation. tkinter — Python interface to Tcl/Tk. Tài liệu chính thức.

4. Python Software Foundation. csv — CSV File Reading and Writing. Tài liệu chính thức.

5. Python Software Foundation. unittest — Unit testing framework. Tài liệu chính thức.

6. Python Software Foundation. time — Time access and conversions. Đồng hồ đo thời gian.

7. Bảng đề tài học phần do người dùng cung cấp. Google Sheets, Topic 12 và yêu cầu bài tập.

Các ví dụ và thiết kế trong báo cáo phục vụ mô hình đã nêu. Kết quả thực nghiệm lấy từ script và file output/, không lấy số liệu minh họa trong tài liệu tham khảo để gán cho chương trình. Trước khi nộp, nhóm cập nhật thông tin thật, kiểm tra link và xem lại các minh chứng từ lần chạy cuối cùng.